



*Ho provato per voi **OpenAI Prism**:
ecco cosa succede quando lo usi su un progetto reale*

Sommario

Ho provato per voi OpenAI Prism: ecco cosa succede quando lo usi su un progetto reale..... 3

1. Premessa 3

2. Introduzione 4

3. Descrizione dell'attività originaria 6

4. Descrizione della review apportata con Prism..... 9

5. Esito prodotto dalla review 12

6. Conclusione 14

7. Appendice..... 16

Ho provato per voi OpenAI Prism: ecco cosa succede quando lo usi su un progetto reale

1. Premessa

Questo articolo nasce da un'esigenza molto concreta: mettere ordine in un'esperienza pratica e, allo stesso tempo, condividerla in modo utile. Negli ultimi mesi ho lavorato alla realizzazione di un prototipo per la previsione del fabbisogno energetico in ambito prosumer, con l'obiettivo di esplorare in modo diretto le potenzialità delle tecniche di machine learning applicate al settore Energy & Utilities. Come spesso accade quando si passa dalla teoria alla pratica, il risultato iniziale – pur funzionante – lasciava aperte alcune domande importanti: quanto è affidabile il modello? Le metriche che sto osservando sono davvero corrette? Il metodo utilizzato è solido oppure sto introducendo, senza accorgermene, errori che falsano i risultati?

È proprio da qui che nasce la prima motivazione di questo contributo: raccontare in modo trasparente la mia esperienza nell'utilizzo di OpenAI Prism. Non un racconto teorico o dimostrativo, ma un caso reale, con tutte le imperfezioni del caso. L'obiettivo non è mostrare “quanto funziona bene uno strumento”, ma piuttosto capire cosa succede quando lo si utilizza davvero, all'interno di un processo di lavoro concreto, con un codice sviluppato in autonomia, con delle scelte progettuali già fatte e con risultati che, almeno inizialmente, non erano del tutto convincenti.

La seconda motivazione è strettamente collegata alla prima: descrivere in modo chiaro e ripercorribile l'attività di revisione del prototipo. Non si tratta semplicemente di “far correggere un codice”, ma di sottoporlo a una vera e propria analisi metodologica. Il passaggio è sottile ma fondamentale. Un modello può funzionare, produrre numeri, restituire metriche apparentemente buone. Ma questo non significa automaticamente che sia corretto dal punto di vista metodologico. Nel mio caso specifico, la revisione ha messo in evidenza una criticità rilevante nella gestione dello scaling e nella valutazione delle prestazioni: un aspetto che, se non affrontato, avrebbe reso i risultati difficilmente interpretabili, se non addirittura fuorvianti.

Ho deciso di inserire questo contributo all'interno della newsletter *Codice Sostenibile* proprio per questo motivo. Chi segue questo spazio sa che, rispetto ad altri contenuti più divulgativi, qui tendo ad entrare maggiormente nel merito delle questioni tecniche. Non per una forma di specialismo fine a sé stesso, ma perché credo che, soprattutto nel momento attuale, sia necessario fare uno sforzo in più per distinguere tra ciò che “sembra funzionare” e ciò che è costruito su basi realmente solide. Nel contesto dell'intelligenza artificiale applicata all'energia, questa distinzione non è secondaria: modelli, previsioni e analisi finiscono spesso per supportare decisioni operative e strategiche.

Questo articolo, quindi, si colloca a metà strada tra racconto esperienziale e analisi tecnica. Da un lato, ricostruisce passo dopo passo il percorso seguito: dalla costruzione del prototipo iniziale, alla prima revisione, fino all'utilizzo di Prism per l'analisi e il miglioramento del modello. Dall'altro, cerca di evidenziare cosa si può imparare da questo processo, anche al di là dello specifico caso d'uso. Non è necessario essere data scientist per coglierne il valore: è sufficiente avere a che fare, anche solo indirettamente, con modelli predittivi o con strumenti basati su AI per capire quanto sia importante la qualità del metodo.

Per chi desidera approfondire gli aspetti più tecnici, ho previsto un'appendice con tutto il materiale di supporto: il codice nelle diverse versioni, la documentazione originale e quella rielaborata, oltre al paper di revisione. Il corpo principale dell'articolo, invece, rimane volutamente più accessibile, con l'obiettivo di rendere il percorso comprensibile anche a chi non ha un background strettamente tecnico ma opera comunque nel settore.

In sintesi, questa non è una dimostrazione di capacità, né una presentazione di risultati "perfetti". È il racconto di un processo: fatto di tentativi, correzioni e miglioramenti. Un processo che, se letto nel modo giusto, rappresenta esattamente ciò che oggi ci si aspetta da un utilizzo consapevole dell'intelligenza artificiale in ambito Energy & Utilities.

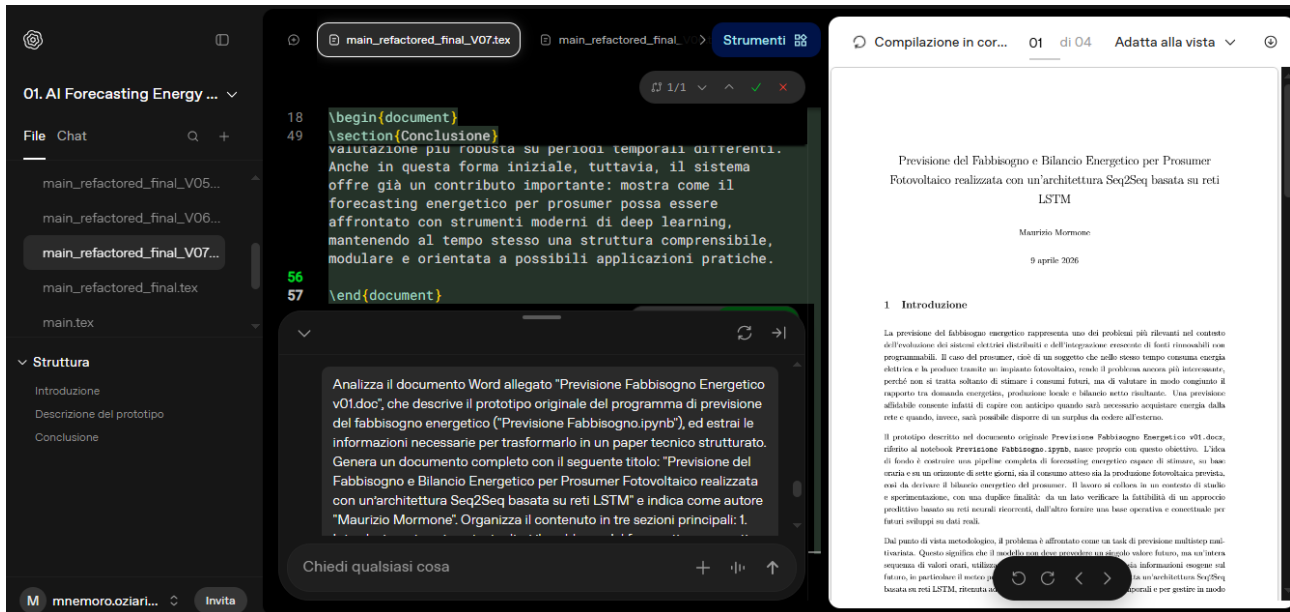
2. Introduzione

Negli ultimi anni abbiamo assistito a un'evoluzione molto rapida degli strumenti basati su intelligenza artificiale. Siamo passati da modelli capaci di generare testo o codice a sistemi in grado di supportare interi flussi di lavoro. In questo contesto si inserisce **OpenAI Prism**, che rappresenta un passo ulteriore: non tanto un "assistente" che risponde a richieste puntuali, ma un ambiente strutturato in cui analizzare, migliorare e documentare progetti complessi.

Per capire davvero cosa sia Prism, è utile partire da ciò che **non è**. Non è semplicemente una chat evoluta (*ChatGPT*), né uno strumento di generazione codice fine a sé stesso (*OpenAI Codex*). Prism è pensato come uno spazio di lavoro in cui è possibile caricare artefatti reali — codice, documenti, notebook — e lavorarci sopra in modo iterativo. Il valore non sta solo nella capacità di generare risposte, ma nel fatto che queste risposte diventano parte di un processo: vengono integrate, versionate, trasformate in documentazione, validate.

Nel mio caso, l'utilizzo è stato molto concreto. Partivo da un notebook Python sviluppato per la previsione dei consumi energetici. Un prototipo funzionante, ma con tutti i limiti tipici di una prima implementazione: scelte progettuali non sempre ottimizzate, alcune semplificazioni, e soprattutto una serie di assunzioni implicite che non erano mai state messe davvero alla prova.

Prism si inserisce esattamente in questo punto del processo: quando hai qualcosa che “funziona”, ma vuoi capire se funziona **nel modo giusto**. (di sotto l'immagine dell'ambiente di lavoro)



Dal punto di vista operativo, il flusso è piuttosto lineare. Si crea un progetto, si caricano i file di interesse e si inizia a interagire. La differenza rispetto ad altri strumenti emerge subito: non si lavora su richieste isolate, ma su un contesto persistente. Il codice che carichi non è solo “letto” una volta, ma diventa parte della memoria operativa del progetto. Questo permette di costruire un dialogo progressivo, in cui ogni richiesta si innesta su quanto fatto in precedenza.

Una delle prime attività tipiche è *l'analisi del codice*. Non nel senso di una semplice descrizione, ma come lettura critica: struttura del modello, flusso dei dati, scelte implementative, eventuali incoerenze. Questo tipo di analisi è particolarmente utile quando si lavora su modelli di machine learning, dove errori apparentemente piccoli — ad esempio nella gestione dei dati o nella pipeline di preprocessing — possono avere un impatto significativo sui risultati finali.

Ma è nella fase successiva che Prism mostra il suo reale valore. Non si limita a individuare problemi, ma è in grado di proporre modifiche concrete. E soprattutto, di farlo in modo strutturato. Le modifiche possono essere integrate direttamente nel codice, mantenendo traccia di ciò che è stato cambiato, oppure trasformate in documentazione tecnica. Questo aspetto è tutt'altro che secondario: spesso il problema non è solo migliorare un modello, ma riuscire a spiegare in modo chiaro cosa è stato fatto e perché.

Un altro elemento interessante è la gestione della documentazione. Prism consente di costruire veri e propri **documenti strutturati** — ad esempio in **formato LaTeX** o **Word** — partendo dall'analisi del codice. Questo permette di passare rapidamente da una fase tecnica a una fase comunicativa, senza dover ricostruire tutto da zero. Nel mio caso,

questo si è tradotto nella produzione di un paper di revisione, in cui venivano descritte sia le caratteristiche del modello sia le criticità individuate e le modifiche proposte.

C'è poi un aspetto meno evidente ma altrettanto importante: Prism introduce una logica di lavoro più simile a quella dei sistemi di versionamento. Le modifiche non vengono applicate automaticamente, ma passano attraverso una fase di revisione e conferma. Questo approccio riduce il rischio di introdurre cambiamenti non desiderati e, allo stesso tempo, rende il processo più consapevole. Non si tratta solo di “accettare una risposta”, ma di validare un intervento.

Dal punto di vista dell'utilizzo quotidiano, questo si traduce in un'interazione continua tra analisi, modifica e verifica. Si parte da una richiesta — ad esempio l'identificazione di possibili criticità — si ricevono indicazioni, si applicano modifiche, si verifica l'impatto sui risultati e si aggiorna la documentazione. Un ciclo che può essere ripetuto più volte, affinando progressivamente il modello.

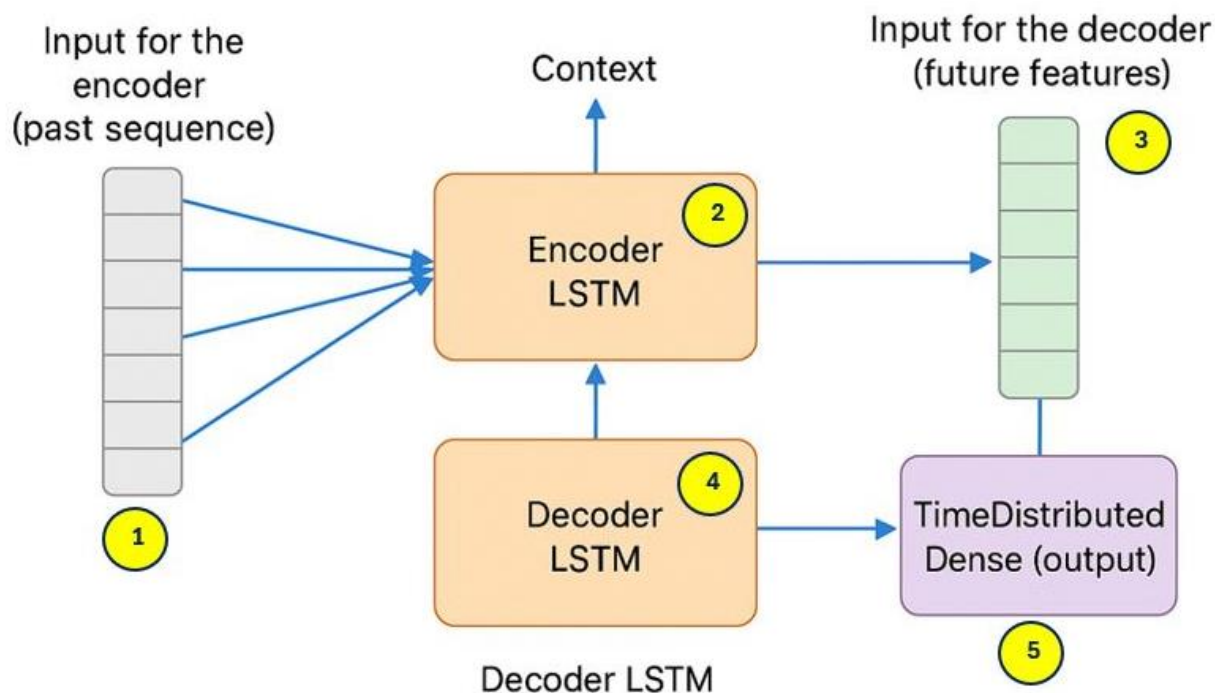
In un contesto come quello dell'Energy & Utilities, questo tipo di approccio ha un valore particolare. I modelli predittivi non sono esercizi accademici, ma strumenti che possono influenzare decisioni operative. Avere la possibilità di analizzarli in modo strutturato, di identificarne i limiti e di migliorarli con un supporto metodologico solido, rappresenta un vantaggio concreto.

In sintesi, Prism non sostituisce il lavoro di chi sviluppa il modello, ma lo accompagna in una fase cruciale: quella in cui **si passa da un prototipo funzionante a una soluzione metodologicamente robusta**. Ed è proprio in questa fase che spesso si gioca la differenza tra un risultato “interessante” e uno realmente utilizzabile.

3. Descrizione dell'attività originaria

L'attività originaria nasce con un obiettivo ben preciso: costruire un prototipo in grado di stimare, in modo coerente e strutturato, il fabbisogno energetico di un prosumer fotovoltaico, integrando in un unico modello sia la componente di consumo sia quella di produzione. Non si trattava quindi di un semplice esercizio di previsione, ma di un tentativo di rappresentare, seppur in forma semplificata, un sistema energetico reale in cui domanda e offerta convivono e interagiscono.

Il punto di partenza è stato uno studio teorico approfondito sul tema del **forecasting energetico**. In particolare, l'attenzione si è concentrata su modelli in grado di catturare la natura temporale dei dati, caratterizzati da stagionalità, trend e forte dipendenza dal contesto esterno. In questo scenario, la scelta è ricaduta su un'architettura di tipo **sequence-to-sequence** basata su **reti LSTM**, ritenuta particolarmente adatta alla gestione di sequenze temporali e alla previsione multi-step.



L'idea alla base del modello è relativamente semplice, ma potente: utilizzare una finestra temporale di dati storici per apprendere le dinamiche passate e combinarla con informazioni future note — come le variabili meteo — per generare una previsione sui giorni successivi. Questo approccio consente di superare i limiti dei modelli puramente autoregressivi, introducendo una componente esogena che, nel caso del sistema energetico, è fondamentale.

Per rendere il prototipo utilizzabile e facilmente testabile, si è scelto inizialmente di lavorare su dati sintetici. Questa scelta, spesso sottovalutata, ha un duplice vantaggio. Da un lato consente di controllare completamente le caratteristiche del dataset — stagionalità, variabilità, rumore — e quindi di verificare se il modello è effettivamente in grado di apprendere le dinamiche desiderate. Dall'altro permette di isolare eventuali problemi metodologici senza l'incertezza legata alla qualità dei dati reali.

Il dataset è stato quindi costruito simulando due grandezze principali: il **consumo energetico** e la **produzione fotovoltaica**. Il **consumo** è stato modellato tenendo conto di pattern giornalieri e settimanali, con variazioni legate a fattori come il giorno della settimana o l'ora del giorno. La **produzione**, invece, è stata fortemente influenzata da variabili meteorologiche simulate, come la radiazione solare e la temperatura, introducendo una dipendenza diretta tra condizioni esterne e output energetico.

Una volta definito il dataset, il passo successivo è stato quello di costruire la **pipeline di preprocessing**. I dati sono stati organizzati in sequenze temporali, suddivise in finestre di input e output. In particolare, per ogni istanza, il modello riceve in ingresso una sequenza di valori storici e restituisce una sequenza di valori futuri. Questo

richiede una trasformazione non banale dei dati, in cui la struttura temporale viene esplicitata e resa compatibile con il modello.

Parallelamente, è stata effettuata una prima normalizzazione delle variabili, con l'obiettivo di rendere il training più stabile. Anche in questa fase sono state fatte alcune scelte progettuali che, come si vedrà successivamente, avranno un impatto non trascurabile sulla qualità dei risultati. Tuttavia, nella fase iniziale, l'obiettivo principale era ottenere un modello funzionante, in grado di apprendere una relazione tra input e output.

L'architettura del modello è stata costruita secondo uno **schema encoder-decoder**. L'**encoder** ha il compito di leggere la sequenza storica e condensarla in una rappresentazione interna, mentre il **decoder** utilizza questa rappresentazione, insieme alle informazioni esogene future, per generare la sequenza di output. I layer LSTM sono stati scelti proprio per la loro capacità di mantenere memoria delle dipendenze temporali, anche su orizzonti relativamente lunghi.

Dal punto di vista implementativo, il modello è stato sviluppato utilizzando librerie standard del mondo Python, con una particolare attenzione alla modularità del codice. Le diverse fasi — generazione dei dati, preprocessing, definizione del modello, training e valutazione — sono state mantenute separate, in modo da facilitare eventuali modifiche o estensioni successive.

Una volta definito il modello, si è passati alla fase di training. Il dataset è stato suddiviso in training set e test set, con l'obiettivo di valutare la capacità del modello di generalizzare su dati non visti. Durante il training, sono state monitorate metriche come la **loss**, mentre in fase di valutazione sono state utilizzate metriche più interpretabili come **MAE** e **RMSE**.

I primi risultati ottenuti erano, a prima vista, incoraggianti. Il modello mostrava una buona capacità di apprendere le dinamiche presenti nei dati sintetici e di produrre previsioni plausibili. Anche le metriche restituite sembravano indicare una performance soddisfacente, soprattutto nella fase di training.

Tuttavia, già in questa fase emergevano alcuni segnali che meritavano attenzione. In particolare, la differenza tra le prestazioni osservate sul training set e quelle sul test set lasciava intuire che qualcosa non fosse del tutto coerente. Le metriche, pur essendo formalmente corrette, non erano immediatamente interpretabili nel contesto reale, soprattutto in termini di unità di misura.

È proprio questo tipo di osservazioni che ha portato alla fase successiva del lavoro. Il **prototipo, così come costruito, rappresentava una base solida, ma non ancora sufficientemente robusta dal punto di vista metodologico**. Le scelte fatte — dalla generazione dei dati alla gestione delle sequenze, fino alla normalizzazione — erano funzionali a ottenere un modello funzionante, ma non necessariamente ottimizzate per garantire la massima affidabilità dei risultati.

In altre parole, l'attività originaria ha raggiunto il suo obiettivo principale: dimostrare la fattibilità dell'approccio e costruire un primo modello operativo. Allo stesso tempo, ha evidenziato la necessità di una fase ulteriore, più orientata alla validazione e alla revisione critica. Ed è proprio da questa consapevolezza che nasce il passaggio successivo, ovvero la decisione di sottoporre il prototipo a un'analisi più approfondita.

4. Descrizione della review apportata con Prism

La fase di review rappresenta il vero punto di svolta dell'intero lavoro. Se l'attività originaria aveva come obiettivo la costruzione di un prototipo funzionante, questa fase ha avuto una finalità diversa: mettere in discussione il modello, verificarne la solidità metodologica e, dove necessario, intervenire in modo strutturato per migliorarlo.

Prima di entrare nel merito delle attività svolte, è utile descrivere brevemente i passaggi iniziali, che costituiscono il punto di ingresso operativo in **OpenAI Prism**.

L'accesso alla piattaforma avviene tramite la **creazione di un'utenza dedicata** (io ho usato l'utenza del mio account Google). Una volta effettuato l'accesso, il primo passo consiste nella **creazione di un nuovo progetto**. Questo elemento è centrale: Prism non è organizzato come una semplice chat, ma come uno spazio di lavoro persistente in cui tutti i file, le interazioni e le modifiche rimangono tracciati e contestualizzati.

All'interno del progetto, è possibile caricare i materiali su cui si intende lavorare. Nel caso specifico, sono stati caricati il notebook Python contenente il prototipo di previsione (**Previsione Fabbisogno.ipynb**) e, successivamente, la documentazione descrittiva del modello. Questo ha permesso di costruire un contesto completo, in cui codice e documentazione potessero essere analizzati congiuntamente.

Una volta impostato l'ambiente, la prima attività richiesta è stata un'**analisi del codice**. L'obiettivo non era semplicemente ottenere una descrizione di ciò che il programma faceva, ma una lettura critica della sua struttura e delle scelte implementative. La richiesta è stata formulata in modo diretto, chiedendo di descrivere il funzionamento del modello e di individuare eventuali punti di debolezza, errori o aree migliorabili.

Un esempio di prompt utilizzato in questa fase è stato:

"Analizza questo programma Python per la previsione dei consumi energetici. Descrivi cosa fa, come è strutturato e individua eventuali errori, criticità metodologiche o punti migliorabili."

Il risultato ottenuto è stato particolarmente interessante perché ha evidenziato, in modo chiaro, alcune criticità che non erano emerse (né in fase di sviluppo, né nella successiva revisione operata da **OpenAI Codex**). In particolare, è stata individuata una potenziale incoerenza nella gestione dello scaling e nella valutazione delle metriche, legata al diverso trattamento dei dati tra fase di training e fase di test.

Questo tipo di analisi rappresenta uno degli aspetti più rilevanti dell'utilizzo di Prism: la capacità di andare oltre la superficie del codice e di individuare problemi metodologici che, pur non generando errori di esecuzione, possono compromettere la qualità dei risultati.

A partire da questa prima analisi, la fase successiva è stata quella di richiedere una **rielaborazione strutturata dei risultati**. In particolare, è stato chiesto di produrre un documento formale che contenesse sia la descrizione del programma sia l'analisi delle criticità individuate. Questo è stato fatto richiedendo la generazione di un documento in **formato LaTeX**, successivamente compilato in PDF.

Il prompt utilizzato è stato del tipo:

“Genera un documento strutturato in due sezioni: una dedicata alla descrizione del programma e una dedicata agli errori e alle possibili migliorie. Utilizza uno stile formale e adatto a un report tecnico. Compila il codice LaTeX e verifica eventuali problemi di formattazione.”

Questo passaggio ha evidenziato un aspetto operativo importante: Prism non si limita a generare contenuti, ma è in grado di trasformarli in documenti strutturati, pronti per essere utilizzati. Tuttavia, l'esperienza ha anche mostrato che è necessario comprendere bene il funzionamento dell'interfaccia per accedere correttamente agli output generati (ad esempio accettando le modifiche o individuando i file prodotti all'interno dell'ambiente).

Una volta ottenuta l'analisi, il passo successivo è stato quello di intervenire direttamente sul codice. In questo caso, l'obiettivo era duplice: da un lato correggere le criticità individuate, dall'altro mantenere traccia chiara delle modifiche apportate. Per questo motivo, è stato richiesto a Prism di generare una nuova versione del programma in cui le modifiche fossero esplicitate in modo trasparente.

Il prompt utilizzato è stato strutturato in modo molto preciso:

“Modifica il programma introducendo le correzioni necessarie. Evidenzia chiaramente le modifiche: racchiudi il codice originale tra delimitatori OLD-COD-INI e OLD-COD-FIN e il nuovo codice tra NEW-COD-INI e NEW-COD-FIN. Genera il risultato in formato notebook Jupyter.”

Questo approccio ha permesso di ottenere un risultato estremamente utile dal punto di vista didattico e operativo: non solo un codice migliorato, ma anche una traccia esplicita di ciò che è stato cambiato e del perché.

Parallelamente alla modifica del codice, è stata condotta una fase di verifica dei risultati. **Le nuove metriche**, calcolate in modo coerente sulla scala originale, **hanno mostrato un miglioramento significativo in termini di interpretabilità e stabilità**. Questo ha confermato che la criticità individuata inizialmente aveva un impatto reale e che la sua correzione era necessaria.

Un altro aspetto rilevante della review ha riguardato la documentazione. Una volta stabilizzata la nuova versione del modello, è stato richiesto a Prism di produrre un documento completo, strutturato come un vero e proprio paper tecnico. Questo documento includeva la descrizione del modello, le modifiche apportate e un'analisi critica delle criticità residue.

Anche in questo caso, il prompt è stato costruito in modo mirato:

“Analizza il notebook aggiornato e genera un documento strutturato in tre sezioni: descrizione del programma, modifiche introdotte e criticità residue. Utilizza uno stile discorsivo, fluido e professionale, evitando elenchi schematici. Fornisci un output pronto per Word.”

Nel corso della revisione, sono state effettuate ulteriori richieste di affinamento della documentazione, tra cui:

- l'inserimento di una sezione di **sintesi delle metriche post-refactoring**
- l'aggiunta di una **descrizione dettagliata dell'architettura del modello**, accompagnata da un diagramma
- la riorganizzazione del documento con abstract, premessa e conclusioni
- la sostituzione di elementi grafici con immagini personalizzate

Questi interventi sono stati guidati da prompt sempre più specifici, con l'obiettivo di ottenere un documento finale non solo corretto dal punto di vista tecnico, ma anche efficace dal punto di vista comunicativo.

Un esempio di prompt utilizzato per arricchire la documentazione è stato:

“Inserisci una sezione di sintesi delle metriche dopo il refactoring dello scaling, includendo una breve interpretazione dei risultati e i principali miglioramenti rispetto alla versione precedente.”

Un altro elemento operativo importante emerso durante la review riguarda la gestione degli output. In alcuni casi, i file generati (ad esempio notebook o documenti) non erano immediatamente visibili o scaricabili. Questo ha reso necessario introdurre richieste esplicite per la creazione di link di download o per la visualizzazione dei file all'interno dell'interfaccia.

Infine, un aspetto che merita di essere sottolineato è il carattere iterativo dell'intero processo.

La review non è stata un'attività "una tantum", ma un ciclo continuo di:

- analisi
- proposta di modifica
- implementazione
- verifica
- documentazione

Questo approccio ha permesso di migliorare progressivamente il modello, mantenendo sempre sotto controllo sia la qualità tecnica sia la coerenza complessiva del lavoro.

In sintesi, la review effettuata con Prism ha trasformato un prototipo iniziale in un modello metodologicamente più solido, accompagnato da una documentazione completa e strutturata. Ma soprattutto, ha reso esplicito un processo che spesso rimane implicito: quello che porta da un'idea funzionante a una soluzione realmente affidabile.

5. Esito prodotto dalla review

La fase di review ha prodotto un risultato che va letto su due livelli distinti ma strettamente collegati: da un lato il miglioramento numerico delle metriche, dall'altro – e soprattutto – la correzione della coerenza metodologica del modello. È proprio questo secondo aspetto a rappresentare il vero valore dell'intervento.

Partendo dai numeri, il confronto tra la versione iniziale del modello e quella successiva alla revisione evidenzia una riduzione molto significativa degli errori. Nel caso del modello di consumo, il MAE sul test passa da valori prossimi a 1.87 a circa 0.19, mentre l'RMSE si riduce da circa 1.89 a 0.23. Una dinamica analoga si osserva sul modello di produzione, dove il MAE scende da circa 0.53 a 0.08 e l'RMSE da circa 0.71 a 0.10. In termini relativi, si tratta di riduzioni dell'ordine dell'85-90%, numeri che, presi isolatamente, potrebbero far pensare a un salto prestazionale molto marcato.

Tuttavia, fermarsi a questa lettura sarebbe fuorviante. Il punto centrale non è tanto il "quanto" le metriche siano migliorate, ma il "perché" siano cambiate in modo così evidente. La revisione ha infatti messo in luce una criticità metodologica rilevante nella gestione dello scaling, che generava una discontinuità tra la fase di training e quella di valutazione. In pratica, **il modello veniva addestrato su dati trasformati, mentre in alcune fasi della validazione e del test le metriche venivano calcolate su scale non perfettamente allineate. Il risultato era una perdita di significato delle metriche stesse, che diventavano difficilmente interpretabili.**

Con il refactoring della pipeline di normalizzazione, questo disallineamento è stato eliminato. Gli scaler sono stati applicati in modo coerente tra training, validation e test, e le metriche finali sono state calcolate riportando le predizioni nella scala originale. Questo passaggio, apparentemente tecnico, ha un impatto decisivo: le metriche tornano ad avere un significato concreto, perché espresse nelle unità fisiche del problema.

Un secondo elemento da evidenziare è la **coerenza tra le diverse fasi di valutazione**. Nella versione rivista del modello, i valori di MAE e RMSE risultano molto simili tra training, validation e test. Per il consumo, ad esempio, il MAE si attesta intorno a 0.17 sul training, 0.19 sulla validation e 0.18 sul test. Lo stesso vale per la produzione, con valori compresi tra 0.07 e 0.08. **Questa uniformità è un segnale importante: indica che il modello è in grado di generalizzare senza mostrare evidenti fenomeni di overfitting.**

Anche questo aspetto va letto nella giusta prospettiva. Nella fase iniziale, le differenze tra training e test non erano solo il risultato del comportamento del modello, ma anche di una pipeline non perfettamente coerente. Dopo la revisione, la stabilità delle metriche diventa un indicatore più affidabile della qualità del modello.

Un altro punto chiave riguarda **l'interpretabilità dei risultati**. Nella versione originaria, pur in presenza di metriche formalmente corrette, era difficile tradurre quei numeri in una valutazione concreta delle prestazioni. Dopo la revisione, invece, è possibile leggere direttamente il significato dell'errore: sapere, ad esempio, che il modello commette un errore medio di circa 0.18 unità sul consumo consente di avere un riferimento chiaro, soprattutto quando si passa a dati reali.

È importante sottolineare che **il miglioramento osservato non è dovuto a una modifica sostanziale dell'architettura del modello**. Non sono stati introdotti layer aggiuntivi, né sono stati stravolti gli iperparametri principali. Il cambiamento è avvenuto a livello di pipeline, ovvero nel modo in cui i dati vengono preparati, trasformati e utilizzati nelle diverse fasi. Questo rafforza un concetto spesso sottovalutato: **nei modelli di machine learning, la qualità del preprocessing e della gestione dei dati è almeno altrettanto importante quanto la complessità del modello stesso.**

Dal punto di vista applicativo, il risultato più rilevante è che il modello, nella sua versione rivista, può essere considerato affidabile per attività di analisi e sviluppo. Le metriche sono coerenti, interpretabili e stabili, e questo consente di utilizzarle come base per ulteriori miglioramenti. Allo stesso tempo, la review ha reso espliciti alcuni limiti ancora presenti.

6. Conclusione

Il percorso di revisione descritto in questo articolo ha permesso di trasformare un prototipo inizialmente utile soprattutto in chiave esplorativa in una versione molto più solida dal punto di vista metodologico. L'analisi condotta con il supporto di **OpenAI Prism** ha avuto un ruolo centrale in questo processo, perché ha consentito di esaminare in profondità la struttura della pipeline, individuare criticità non immediatamente evidenti e ricondurre l'intero flusso di lavoro a criteri più rigorosi di coerenza tra preprocessing, addestramento e valutazione.

Il punto più delicato è emerso nella gestione dello scaling del target. Nella configurazione iniziale, questo aspetto introduceva un'ambiguità significativa nell'interpretazione delle metriche, rendendo difficile comprendere il reale livello di accuratezza del modello. La revisione ha permesso di intervenire in modo mirato su questo nodo, riportando ordine e coerenza in una fase cruciale del processo.

L'intervento non si è limitato a una correzione puntuale, ma ha portato a un vero e proprio refactoring della pipeline. La separazione tra training, validation e test è stata resa più chiara e metodologicamente corretta; il fit degli scaler è stato ricondotto esclusivamente ai dati di training; la valutazione finale è stata riportata nella scala fisica originale delle grandezze previste. Questo ha permesso di ottenere metriche finalmente interpretabili e affidabili, non tanto perché "migliori" in senso assoluto, ma perché coerenti con il significato operativo del problema.

Ed è proprio questo il risultato più rilevante dell'intero lavoro. Il modello, nella sua versione attuale, non è semplicemente più performante: è soprattutto più credibile. Le metriche possono essere lette e comprese, il comportamento del modello è più trasparente e la coerenza tra le diverse fasi del processo consente di utilizzarlo come base per ulteriori sviluppi. In altre parole, **si è passati da un sistema che funzionava a un sistema che può essere valutato in modo consapevole.**

Resta naturalmente il carattere prototipale del lavoro. L'utilizzo di dati sintetici, l'assenza di benchmark comparativi e una limitata varietà di feature rappresentano elementi che richiedono ulteriori sviluppi. Tuttavia, la revisione ha costruito un fondamento molto più solido su cui lavorare. È una differenza sostanziale: migliorare un modello già metodologicamente coerente è un'attività incrementale; correggere un modello con basi fragili è un'operazione molto più complessa.

Guardando in prospettiva, le evoluzioni più interessanti riguardano l'introduzione di dati reali, il confronto sistematico con modelli baseline più semplici, l'arricchimento delle variabili esplicative e una gestione più realistica dell'incertezza, in particolare quella legata alle previsioni meteorologiche. Sono passaggi necessari per avvicinare il modello a un utilizzo operativo, soprattutto in un contesto come quello dell'Energy & Utilities, dove l'affidabilità delle previsioni ha un impatto diretto sulle decisioni.

Nel complesso, **il lavoro svolto non rappresenta soltanto una revisione tecnica, ma un cambio di prospettiva. Sposta l'attenzione dal risultato in sé al modo in cui quel risultato viene ottenuto.** Ed è proprio in questo passaggio che si coglie il valore dell'intero percorso: non tanto nell'aver costruito un modello più accurato, ma nell'aver reso il processo che lo sostiene più rigoroso, trasparente e, soprattutto, riutilizzabile.

7. Appendice

Di sotto il materiale studio disponibile

File	Descrizione	Origine	Origine	Disponibile	Formato
Previsione Fabbisogno v05	Programma originale di Previsione consumi scritto in Python	Google Drive	GitHub		notebook jupiter
Previsione Fabbisogno v05_refactored_final v03	Programma di Previsione consumi revisionato da Prism	Google Drive	GitHub		
Previsione Fabbisogno Energetico v01	Documentazione realizzata da me del programma originario	Google Drive	GitHub		PDF
00. Previsione Fabbisogno Energetico v01	Documentazione del programma originario realizzata da Prism in formato Paper scientifico	Google Drive	GitHub	Linkedin	
02. AI Forecasting Energy Consumption_refactored_final_V05	Documentazione della review effettuata da Prism sul programma originario e realizzata in formato Paper scientifico	Google Drive	GitHub	Linkedin	